

C

```
#include <stdio.h> // header file

int main(){

    printf("Hello world \n");

    // variables, data types

    // 1 gb => 1024 mb
    // 1 mb => 1024 kb
    // 1kb => 1024 byte
    // 1 byte => 8 bit
    // 1 bit => either 0 or 1

    int a = 10; // => 4 bytes

    float b= 20;

    double d = 30;

    char c = 'F';
    int r; // int default = > 0

    printf("%d \n" , r);

    printf("%d \n" , a);

    printf("%.2f \n" , b);

    printf("%Lf \n" , d);

    printf("%c \n" , c);


    return 0;
}
```

Day2.c

```
#include <stdio.h>
#include <stdbool.h>

int main(){

    // int a;

    // printf("Enter a Integer number: ");
    // scanf("%d", &a);

    // printf("%d\n", a);

    // float b;

    // printf("Enter a Float number: ");
    // scanf("%f", &b);

    // printf("%.2f\n", b);

    // char c;

    // printf("Enter a character: ");
    // scanf("%c", &c);

    // printf("%c \n", c);

    // operators
    // it's a special symbol that's tells compiler
    // to do a specific task

    // unary, binary, ternary

    // unary => need only one operand

    // ++, --

    int count = 0;

    count++; //1
    count--; // 0

    count++; // 1
    count++; // 2
```

```

count--; // 1

// printf("%d \n", count);

//binary => need atleast two operand
// arithmetic => (+, -, /, *, %)

int x = 10;
int y = 10;

// int result = x + y;
// result = x-y;
// result = x*y;
// result = x/y;

// result = 3 % 2 ;

// printf("%d \n", result);

// relational => (> , <, >=, <=, ==, != )

// 10 > 4
// printf("%d \n", x > y);
// printf("%d \n", x < y);

// printf("%d \n", x >= y);

// printf("%d \n", x <= y);

// printf("%d \n", x == y);

// printf("%d \n", x != y);

// ramyasaravana05@gmail.com
// 6385705108

// logical ( && , ||, !)

// printf(" AND %d \n", true && false);
// true true => true
// false true => false
// false false => false

// printf("OR %d \n", false || true);

// flase fasle => false

```

```

    // true false => true

    // printf("NOT %d \n", !false);

    // assignment (+=, -=, /=, *=, %=)

    int mark = 85;

    // mark = mark + 5;

    mark += 5;

    // bitwise (bit => 0 or 1)
    // & | ^ >> <<

    int a = 5;
    int b = 6;

    // printf("%d \n", a & b);
    // printf("%d \n", a | b);
    // printf("%d \n", a ^ b);

    // >> => right shift operator
    printf("Right shift : %d \n", a >> 1);

    printf("Left shift : %d \n", a << 1);

    // const double PI = 3.14;
    // PI = 3.22;
    // printf("%.2Lf \n", PI);

    // bool isEligible = true;

    // printf("%d \n", isEligible);

    return 0;
}

```

Day3.c

```

#include <stdio.h>
#include <stdbool.h>

```

```

int main(){

    //control structures
    printf("Control structures \n");

    // conditional statements ( if, if-else, else-if, nested if , switch)

    int age = 18;

    bool voterId = false;

    // if(age > 18){
    //     printf("You are eligible to vote \n");
    // }else{
    //     printf("You are not eligible \n");
    // }

    // nested if

    // if(age >= 18){

    //     if(voterId){
    //         printf("You are eligible to vote \n");
    //     }else{
    //         printf("you don't have voter id");
    //     }

    // }else{
    //     printf("Your age is less than 18\n");
    // }

    // else-if ladder
    int marks = 90;
    // if( marks >= 90){
    //     printf("A \n");
    // }else if(marks >= 80){
    //     printf("B \n");
    // }else if(marks >= 70){
    //     printf("C \n");
    // }else {
    //     printf("Fail \n");
    // }

    switch(marks){
        case 90:
            printf("A \n");
            break;
        case 80:
            printf("B \n");

```

```

        break;
    default:
        printf("Fail \n");
        break;
}
// llashveinram@gmail.com
// 8778100220

// loops (for, while, do-while)

return 0;
}

```

Day4.c

```

#include <stdio.h>

int main(){

    printf("Hello there \n");

    //loops

    // for, while, do-wihle

    // for(int i=1; i<=10 ; i++ ){
    //     // if(i % 2 == 0){
    //         printf("Even number: %d \n", i);
    //     }
    //     printf(" %d \n", i);
    // }

    //while loop
    // int j=10;
    // while(j>0){
    //     printf("J value : %d \n", j);
    //     j--;
    // }

    // do-while loop

    // int x = 1;
    // do{
    //     printf("%d \n", x);
    // }
    // while(x<10);
}

```

```

//      x++;
// }while(x == 0);

// jump statement => break , continue

// for(int i=1; i<=10; i++){

//      if(i == 5){
//          // break;
//          continue;// skips a current iteration
//      }
//      printf("%d \n", i);

// }

// array

// int numbers[] = {10,20,30,40,50};

// printf("%d \n", numbers[0]);

// printf("%d \n", numbers[3]);

// array => for loop

// numbers[2] = 60;

// for(int i=0; i<5; i++){
//     printf("%d \n", numbers[i]);
// }

int size;
printf("Enter array size: ");
scanf("%d", &size);

int useArray[size];

for(int i=0; i<size; i++){
    printf("Enter %d th value: ",i );
    scanf("%d ", &useArray[i]);
}

int sum;

for(int i=0; i<size; i++){
    sum += useArray[i];
    printf("%d \n", useArray[i]);
}

```

```

    }

    printf("Sum value %d", sum);

    return 0;
}

```

Day5.c

```

#include <stdio.h>
#include <string.h>

int main(){
    char str[] = "FIIT";
    char name[40];
    strcpy(name , str);
    printf("Strcpy method %s \n", name);
    printf("%s\n", str);
    printf("%d \n", strlen(str));
    char str1[] ="Hello";
    char str2[] = "Hello";
    // strcat(str1 ,str2);
    printf("%s \n", str1);
    printf("Compare method %d \n", strcmp(str1, str2));

    int arr[] ={1,2,3,4,5};

    int arr2d[3][3] = { {1,2,3} , {4,5,6} , {7,8,9} };

    int matrix[3][3] = { {1,2,3} , {4,5,6} , {7,8,9}};

    int result [3][3];

    int sum ;
    for(int i=0; i<3; i++){ // row

        for(int j=0; j<3; j++){
            sum += arr2d[i][j];

            result [i][j] = arr2d[i][j] + matrix[i][j];
        }
    }

    printf("Sum value of arr2d %d \n", sum);
    for(int i=0; i<3; i++){ // row

        for(int j=0; j<3; j++){

```

```

        printf("%d ",result[i][j]);
    }

    printf("\n");

}
// 1 2 3
// 4 5 6
// 7 8 9

printf("%d \n", arr[0]);

printf("%d \n", arr2d[2][0]);

for(int i=0; i<3; i++){ // row

    for(int j=0; j<3; j++){

        printf("%d ",arr2d[i][j]);
    }

    printf("\n");

}

return 0;
}

```

Day6.c

```

#include <stdio.h>
#include <string.h>
// with argument and with return type
int add(int x, int y){ // parameter
    int sum = x+y;
    return sum;
}
// without argument and without return type
void display(){ // function definition
    printf("Display method \n");
}
// with argument and without return type
void multiply(int a , int b, int c){

    printf("%d \n", a*b*c);
}

```

```

}
// without argument and with return type
double PI(){
    double pi = 3.14;
    return pi;
}

struct Student{
    int roll_no;
    char name[50];
    double marks;
};

int main(){
    struct Student classroom[5] = {
        {1,"Rithish", 90},
        {2,"lashveinram" ,95},
        {3, "raj", 80}
    };

    for(int i=0; i<3; i++){
        printf("%d %lf %s \n", classroom[i].roll_no, classroom[i].marks ,
classroom[i].name);
    }

    struct Student s2 = {20, "Redmi", 80};
    struct Student s1;
    s1.roll_no = 21;
    s1.marks = 90;
    strcpy(s1.name , "FIIT");

    printf("%d \n", s1.roll_no);
    s1.marks = 100;
    printf("%lf \n", s1.marks);
    printf("%s \n", s1.name);

    printf("%d \n", s2.roll_no);
    printf("%lf \n", s2.marks);
    printf("%s \n", s2.name);

```

```

// int a = 10;
// int *ptr = &a;

// int **dptr = &ptr;

// printf("Double pointer : %d \n", **dptr);

// printf("%p \n", ptr);
// printf("%d \n", *ptr);
// int arr[] = {1,2,3,4,5};
// int *index = arr;
// for(int i=0; i<5; i++){
//     // printf("Pointer %d and i value :%d \n", *index, i);
//     printf("%d \n", *(index + i));
// }
// index++;
// index++;
// printf("%d \n", *index);
// index--;
// printf("%d \n", *index);

// double pi = PI();

// printf("%.2lf \n",pi);

// multiply(10,20,30);

// printf("Functions \n");

// int a =10;
// int b = 20;

// int total = add(a, b); // argument
// printf("%d \n", total);

// display(); // function call

return 0;
};

```

Theory

1. Variables & Data Types

- **int** → 4 bytes, stores integers.
- **float** → 4 bytes, single precision decimal.
- **double** → 8 bytes, double precision decimal.
- **char** → 1 byte, stores single character.

2. Operators

- **Unary** → ++, --.
- **Binary** → Arithmetic (+, -, *, /, %), Relational (>, <, ==, !=), Logical (&&, ||, !).
- **Assignment** → +=, -=, *=, /=, %=.
- **Bitwise** → &, |, ^, >>, <<.

3. Control Structures

- **if, if-else, else-if, nested if** → decision making.
- **switch-case** → multiple branch selection.

4. Loops

- **for loop** → definite iteration.
- **while loop** → indefinite iteration.
- **do-while loop** → executes at least once.
- **break/continue** → jump statements.

5. Arrays & Strings

- **1D array** → int arr[5].
- **2D array** → int arr[3][3].
- **String functions** → strcpy, strlen, strcmp, strcat.

6. Functions

- **Types:**
 - With argument & return type → int add(int a, int b).
 - With argument & no return → void multiply(int a, int b).
 - No argument & return type → double PI().
 - No argument & no return → void display().

7. Structures

- **Definition:** User-defined data type grouping related variables.
- Example:

```
struct Student {  
    int roll_no;  
    char name[50];  
    double marks;  
};
```

8. Pointers

- **Definition:** Variable that stores memory address.
- **Dereferencing:** *ptr gives value at address.
- **Double pointer:** **dptr points to pointer.
- **Array with pointers:** *(arr+i) accesses elements.



Coding Round Questions

- Data types → What are basic data types in C?
- Operators → Explain bitwise operators with examples.
- Control structures → Difference between if-else and switch.
- Loops → Explain for, while, and do-while.
- Arrays → How do you declare and access arrays?
- Strings → Explain strcpy, strlen, strcmp.
- Functions → What are the four types of functions in C?
- Structures → How do structures differ from arrays?
- Pointers → What is a pointer and how is it used?

Machine Coding Round Challenge

Problem: Student Records with Functions & Structures

Requirements:

1. Create a struct Student with fields: roll_no, name, marks.
2. Write functions to:
 - Add student details.
 - Display student details.
 - Calculate average marks.
3. Use an array of structures to store multiple students.
4. Demonstrate pointer usage to access student data.